

Yes. For Python datetime, these are the **standard strftime() / strptime() format codes** you should know.

Date format codes

Code	Meaning	Example
%Y	4-digit year	2026
%y	2-digit year	26
%m	Month 01–12	09
%B	Full month name	September
%b	Short month name	Sep
%d	Day of month 01–31	10
%j	Day of year 001–366	253
%U	Week number, Sunday first	36
%W	Week number, Monday first	36
%w	Weekday number Sunday=0	4
%a	Short weekday name	Thu
%A	Full weekday name	Thursday

Time format codes

Code	Meaning	Example
%H	Hour, 24-hour 00–23	15
%I	Hour, 12-hour 01–12	03
%M	Minute 00–59	18
%S	Second 00–59	42
%f	Microseconds	123456
%p	AM / PM	PM
%z	UTC offset	+0530
%Z	Time zone name	IST

Useful special codes

Code	Meaning	Example
%c	Local date + time	Thu Sep 10 15:18:42 2026
%x	Local date	09/10/26
%X	Local time	15:18:42

Most important ones to memorize

For everyday Python development, start with these:

%Y → 2026

%m → 09

%d → 10

%H → 15

%I → 03

%M → 18

%S → 42

%p → PM

%A → Thursday

%B → September

Example

```
from datetime import datetime
```

```
now = datetime.now()
```

```
print(now.strftime("%d-%m-%Y"))
```

```
print(now.strftime("%d/%m/%Y"))
```

```
print(now.strftime("%A, %d %B %Y"))
```

```
print(now.strftime("%I:%M:%S %p"))
```

```
print(now.strftime("%H:%M:%S"))
```

Output could be:

10-09-2026

10/09/2026

Thursday, 10 September 2026

03:18:42 PM

15:18:42

The big distinction

`strftime()`

datetime → **string**

```
now.strftime("%d-%m-%Y")
```

Whereas:

`strptime()`

string → **datetime**

```
datetime.strptime("10-09-2026", "%d-%m-%Y")
```

One small gotcha: %m is **month**, while %M is **minute**. That's a classic beginner trap.